

Unit III

Software Design, Testing Strategy & Product Metrics

Introduction

Software design is one of the most critical phases of the **Software Development Life Cycle (SDLC)**. It involves **creating a blueprint** that defines how the software system will be structured and implemented to satisfy all requirements mentioned in the **Software Requirements Specification (SRS)** document. The design process converts the "what to do" (from requirement analysis) into "how to do" (implementation plan)..

Objectives of Software Design

The main goals of software design are:

- To translate requirements into a structured solution.
- To create a system that is **reliable, maintainable, efficient, and scalable**.
- To provide a **clear blueprint** for programmers and developers.
- To **minimize complexity** by using design principles like modularity and abstraction.
- To ensure that design supports **future modifications** and **reuse**.

Levels of Design

Software design is usually divided into two major levels:

1 Architectural Design (High-Level Design): Focuses on the overall system structure. Defines major modules, their interconnections, and data flow. It identifies system architecture styles (e.g., layered, client-server, MVC). Example: Identifying modules like *User Interface, Business Logic, and Database Layer*.

- **Output:** System architecture diagrams, module interactions, data flow diagrams (DFD).

2 Detailed Design (Low-Level Design): Deals with the internal logic and working of each module. Specifies algorithms, data structures, classes, interfaces, and database schema. Ensures that each module fulfills its intended functionality.

- **Output:** Pseudo code, flowcharts, class diagrams, ER diagrams, etc.

Software Design Process

1. **Requirement Analysis** - Review the SRS to understand what the system must do.
2. **System Structuring** - Divide the system into subsystems and modules.
3. **Data Design** - Identify and define the data structures and databases needed.
4. **Architectural Design** - Define overall system architecture (how modules interact).
5. **Interface Design** - Design user interfaces and module-to-module interfaces.
6. **Algorithm Design** - Define algorithms for processing and logic implementation.
7. **Design Review and Evaluation** - Assess design for completeness, correctness, and quality.
8. **Documentation** - Prepare the Software Design Document (SDD).

Design Quality

2.1 Definition

Design quality refers to **how well the software design fulfills its purpose**, meets user and system requirements, and supports future changes with minimal cost and effort.

High-quality software design ensures that the system is:

- **Reliable** (performs correctly),
- **Maintainable** (easy to modify),
- **Reusable** (can be used in other systems),
- **Efficient** (uses resources optimally).

Factors Affecting Design Quality

Factor	Description
Correctness	Design should implement all functional requirements accurately.
Understandability	The design must be easy to read and comprehend for future maintenance.
Efficiency	Should make optimal use of memory, CPU time, and resources.
Modularity	The system should be divided into independent, cohesive modules.
Reusability	Modules and components should be usable in other projects.
Flexibility	Design should easily accommodate changes or enhancements.

Factor	Description
Maintainability	Easy to locate and fix errors or update functionality.
Portability	Design should be adaptable to various hardware or software environments.
Security (Integrity)	Protects data and code from unauthorized access or damage.
Robustness	Ability to handle unexpected errors or inputs gracefully.

Characteristics of a Good Design

A good software design should:

- Meet all functional and non-functional requirements.
- Exhibit simplicity, avoiding unnecessary complexity.
- Be structured and modular.
- Promote consistency in naming, documentation, and layout.
- Be testable and traceable — each module should be verifiable.

Key Design Concepts

The following are **fundamental principles and concepts** that guide the design process and ensure design quality.

1 Abstraction

- **Definition:** The process of focusing on essential characteristics and ignoring irrelevant details.
- **Purpose:** Simplifies complex systems by modeling them at different levels.
- **Types of Abstraction:**
 - **Functional Abstraction:** Specifies *what a function does*, not *how it works*.
 - **Data Abstraction:** Defines *data objects* and hides details of how data is stored.
 - **Control Abstraction:** Manages control mechanisms like iteration, conditionals, etc.

2 Modularity

- **Definition:** Dividing the software into smaller, independent, and manageable units called **modules**.
- **Advantages:**
 - Easier debugging and testing.

- Promotes reusability.
 - Improves maintainability and scalability.
- Each module should have:
 - **High Cohesion** - performs one specific task.
 - **Low Coupling** - minimal dependency on other modules.

3 Encapsulation

- **Definition:** The process of bundling data and methods operating on that data within a single unit (like a class in OOP).
- Prevents direct access to data and promotes data integrity.
- Example: In Java, declaring class variables as private and providing getter/setter methods.

4 Information Hiding

- Hides the internal workings of modules and exposes only necessary interfaces.
- Reduces complexity and interdependency between modules.
- Example: A user doesn't need to know how a sorting function works, only how to call it.

5 Cohesion and Coupling

- **Cohesion:** Degree to which the elements of a module are related to one another.
 - **High Cohesion** → better module design.
 - Types include Functional, Sequential, Communicational, and Logical cohesion.
- **Coupling:** Degree of dependency between modules.
 - **Low Coupling** → better independence.
 - Types include Data, Control, and Common coupling.

6 Separation of Concerns

- Each module should handle a **single, well-defined concern**.
- Helps in reducing complexity and making the system more maintainable.
- Example: In MVC architecture — Model (data), View (interface), Controller (logic).

7 Refinement: The process of elaborating a high-level design into more detailed specifications.

- A top-down approach, starting from abstract system design to concrete implementation details.

8 Software Architecture

- Describes the overall structure of the system: how components are organized and how they interact.
- Architectural styles include:

Layered architecture: The Layered Architecture (also called **n-tier architecture**) is one of the most widely used architectural styles in software design. It organizes software into layers, where each layer has a specific role and responsibility, and interacts only with the layers directly above or below it. This structure helps in achieving **modularity, maintainability, scalability, and reusability**.

Client-server architecture: The Client-Server Architecture is a distributed system model that divides an application into two main components — the client and the server — which communicate over a network. This architecture is fundamental in network-based applications such as web applications, email systems, and databases.

Micro services: Micro services Architecture is a modern software design approach in which a large application is divided into a collection of small, independent services. Each service performs a specific business function, runs in its own process, and communicates with other services through lightweight APIs (usually HTTP/REST or messaging queues).

Pipe-and-filter: Pipe-and-Filter Architecture is a data-flow based architectural style commonly used in systems that process data in a sequence of transformations.

In this architecture, the system is divided into a set of components (filters) that transform data, and connectors (pipes) that pass data from one filter to another.

Event-driven: Event-Driven Architecture (EDA) is a software design pattern in which the flow of the program is determined by events — such as user actions, sensor outputs, or messages from other programs or systems.

9 Design Patterns: Reusable templates or solutions to common design problems.

- **Examples:**
 - **Creational Patterns:** Singleton, Factory.
 - **Structural Patterns:** Adapter, Composite.
 - **Behavioral Patterns:** Observer, Strategy.

10 Design Documentation: Design should be properly documented to serve as a reference for development and maintenance. Common documents are **Software Design Document (SDD)**, **Data Dictionary & UML Diagrams (Use Case, Class, Sequence, Activity)**

Data and Architecture Design

Data and architecture design are two crucial phases in the software design process. They define how the system will be structured, how data will be organized, and how components will interact to meet both functional and non-functional requirements (like scalability, performance, and maintainability).

Data Design : Data design focuses on how data is stored, organized, and accessed within the system.

Objectives of Data Design

- Ensure data integrity, security, **and** consistency.
- Enable efficient data access, storage, **and** retrieval.
- Support scalability **and** future data growth.
- Provide **a** logical and physical data model for implementation.

Steps in Data Design

1. **Identify Entities and Relationships**
 - Determine major data objects (e.g., Student, Course, Teacher).
 - Identify relationships (e.g., Student-Enrolls-Course).
2. **Create Data Models**
 - **Conceptual Model:** High-level ER (Entity-Relationship) diagram.
 - **Logical Model:** Defines attributes, keys, and relationships (normalized).
 - **Physical Model:** Maps logical design to physical storage (tables, indexes, partitions).
3. **Normalize the Data**
 - Apply **normalization** to remove redundancy and maintain consistency.
4. **Define Data Storage Strategy**
 - Choose storage type (relational DB, NoSQL, file system, etc.).
 - Plan for **backups, replication, and archiving**.

5. Data Flow Design

- Represent how data moves across the system using **Data Flow Diagrams (DFD)**.

3. Architecture Design

Architecture design defines the **overall structure of the software system** — how modules interact, communicate, and cooperate.

Objectives of Architecture Design

- Organize the system into **layers, modules, or services**.
- Promote **modularity, maintainability, and reusability**.
- Ensure **separation of concerns** (each part handles a distinct responsibility).
- Support **scalable and fault-tolerant** operation.

Key Architectural Principles

a) Modularity

- Divides a system into **independent modules** that can be developed and maintained separately.
- Each module performs a specific function (e.g., authentication, payment, reporting).
- Benefits: easier testing, maintenance, reusability, and parallel development.

b) Separation of Concerns

- Each module or layer focuses on **one responsibility** only.
- Example:
 - **UI Layer** → handles presentation
 - **Business Layer** → processes logic
 - **Data Layer** → manages storage and retrieval
- Reduces complexity and improves clarity.

c) Scalable Architecture

- Designed to handle **growing users, data, and load** without performance degradation.
- Two main scaling strategies:
 - **Vertical Scaling**: Add more power (CPU/RAM) to existing machines.
 - **Horizontal Scaling**: Add more machines (servers) and distribute the load.

- Common methods: caching, load balancing, database sharding, and micro services.
-

Unified Modeling Language (UML) and Structural Modeling Techniques

The Unified Modeling Language (UML) is a standard visual modeling language used in software engineering to represent, design, and document the architecture and behavior of complex systems. It was developed to unify the different notations and methods previously used in object-oriented design, such as Booch, OMT, and Objectory.

UML is not a programming language, but a visual notation that helps developers, analysts, and stakeholders to understand and communicate system requirements and design clearly before actual coding begins.

Objectives of UML

- To provide a standard way to visualize the system's design.
- To simplify complex systems through graphical representation.
- To serve as a blueprint for system construction.
- To improve communication among project members.
- To support different phases of software development — from requirement analysis to implementation and maintenance.

Major Features of UML

- It is independent of any specific programming language.
- Provides both structural (static) and behavioral (dynamic) views.
- Encourages object-oriented modeling through abstraction, encapsulation, and inheritance.
- Promotes reusability and scalability of system components.

Types of UML Diagrams

UML provides a collection of 14 different diagrams categorized into two main types:

A. Structural Diagrams (Static View)

Structural diagrams describe the **static structure** of the system — its classes, objects, components, and the relationships among them. They show **what the system is**, not how it behaves.

1. Class Diagram

- Represents classes, their attributes, operations (methods), and relationships.
- It is the **core of object-oriented modeling**.
- Example relationships: association, generalization (inheritance), dependency, aggregation, composition.

2. Object Diagram

- Shows the **snapshot** of the system at a particular moment.
- Depicts **instances of classes** (objects) and their values.
- Used to verify the accuracy of class diagrams.

3. Component Diagram

- Describes **organization and interdependencies** among software components.
- Useful during **implementation phase** to understand modular structure.

4. Deployment Diagram

- Represents the **physical deployment** of software artifacts on hardware nodes.
- Useful for system administrators and network designers.

5. Package Diagram

- Organizes UML elements into **packages** (logical grouping of classes).
- Used to manage large and complex systems.

6. Composite Structure Diagram

- Shows **internal structure** of a class and collaboration among its parts.
- Helps model complex components within a single class or subsystem.

B) Behavioral Diagrams (Dynamic View): Behavioral diagrams describe **how the system behaves** — the interactions among components and the changes in their states over time.

Major behavioral diagrams include:

- **Use Case Diagram** - Describes system functionality from the user's point of view.
- **Sequence Diagram** - Shows the sequence of interactions among objects over time.
- **Activity Diagram** - Represents the workflow of operations.
- **State Machine Diagram** - Depicts changes in object states in response to events.

Structural Modeling Techniques: Structural modeling refers to techniques used to represent the static structure of a system in terms of objects, classes, attributes, relationships, and packages. It focuses on how the system is organized rather than how it behaves.

The structural model defines the foundation of the software system upon which behavior and interactions are built.

Objectives of Structural Modeling

- To define the framework and architecture of the system.
- To represent objects and their relationships clearly.
- To support modularity and reusability.
- To assist in database and code generation later in the development cycle.

Key Elements of Structural Modeling

1. Classes and Objects

- **Class:** Blueprint that defines the structure and behavior (attributes and operations) of objects.
Example:
Class Student → Attributes: RollNo, Name; Operations: register(), attendExam().
- **Object:** Instance of a class that holds actual data.
Example: Student1 (instance of class Student).

2. Attributes and Operations

- **Attributes:** Properties or data members that define the state of a class (e.g., name, age, salary).
- **Operations:** Functions or methods that define the behavior (e.g., calculateGrade(), payFees()).

Packages

- Used to **group logically related classes or components** into manageable modules.
- Promotes **encapsulation** and **layered design**.

Structural Modeling Techniques Examples

1. Class Diagram Technique

- Identifies key classes and their relationships.

1. Entity-Relationship (ER) Modeling

- Used for **database design** to show entities, attributes, and relationships.
- Example: Student (Entity) — Enrolls — Course (Entity).

2. Component and Deployment Models

- Focus on physical structure and how different components interact and are deployed in real systems.

Advantages of Structural Modeling

- Provides a **clear, static view** of system architecture.
- Enhances **system understanding and communication**.
- Serves as a **foundation** for behavior modeling and implementation.
- Helps identify **dependencies and potential design flaws** early.
- Supports **modular and scalable system design**.

Relationship between UML and Structural Modeling

Aspect	UML	Structural Modeling
Definition	A visual modeling language for representing all aspects of a system.	A method focusing on static structure (classes, objects, relationships).
Focus	Both static and dynamic aspects.	Only static architecture and design.
Key Diagrams	Class, Object, Component, Deployment.	Class and ER diagrams mainly.
Purpose	System visualization and documentation.	System architecture definition.
Outcome	Complete design blueprint.	Static model for further design and implementation.

Software Testing Strategies

Software Testing is a systematic process of **evaluating a software system** to ensure that it meets the specified requirements and is **free from defects or errors**.

It is not only about finding bugs but also about **verifying that the software works correctly and efficiently** under various conditions.

A **testing strategy** defines the **overall approach and plan** used to test the software throughout the **Software Development Life Cycle (SDLC)**. It ensures that testing is conducted in a **structured, efficient, and cost-effective** manner.

Objectives of Software Testing

- To ensure that the software system **meets functional and non-functional requirements**.
- To identify and correct **defects** before the product is released.
- To **improve software quality** and reliability.
- To validate that the system works correctly in **different environments**.
- To ensure **customer satisfaction** by delivering a defect-free product.

Excellent — here's a **clear, detailed, and well-organized explanation of Conventional Testing, Black Box Testing, and White Box Testing**, suitable for college notes or teaching material ☺

Conventional Testing, Black Box Testing, and White Box Testing

Introduction to Software Testing

Software Testing is the process of **verifying and validating** that a software product or application:

- Meets the specified requirements,
- Works as expected,
- And is free of defects.

Testing ensures **quality, reliability, and correctness** of the software product before delivery.

Testing approaches can be broadly classified as:

- **Conventional Testing**
- **Unconventional (Modern / Structural) Testing**
(e.g., Black Box, White Box, and Gray Box Testing)

Conventional Testing: Conventional testing refers to the traditional approach to software testing that focuses on detecting errors after the coding phase of the software development life cycle (SDLC).

It is also known as **testing after implementation**, where the emphasis is more on **error detection rather than prevention**.

Characteristics of Conventional Testing:

- Testing begins **only after the code is developed**.
- Mainly involves **execution-based testing**.
- Focus is on finding defects rather than preventing them.
- Limited involvement of testers during design and development phases.
- Typically used in the **Waterfall Model** of SDLC.

Phases in Conventional Testing Approach:

1. **Requirements and Design Phases:**
 - Limited or no testing is performed here.
2. **Coding Phase:**
 - Developers write the code.
3. **Testing Phase:**
 - Testers perform testing after the code is complete.
 - Includes
 - **Unit testing:** Focuses on testing **individual units (functions, modules, or classes)** of code in isolation. Ensures that each part of the software performs as expected. Typically automated using tools like **JUnit (Java)**, **PyTest (Python)**, or **NUnit (.NET)**.

Advantages: Detects errors early in development and Simplifies debugging and maintenance.

Integration testing: After unit testing, integration testing ensures that **modules interact correctly** when combined.

Common Integration Strategies:

1. Big-Bang Integration:

- All modules are integrated and tested at once.
- Simple but difficult to debug errors.

2. Incremental Integration:

- Modules are integrated and tested one by one.
- Easier to isolate and fix defects.

Types:

- **Top-Down Integration:** Start testing from the top module (main control) and integrate downward using stubs.
- **Bottom-Up Integration:** Start from the lower modules and integrate upward using drivers.
- **Sandwich (Hybrid) Integration:** Combines both top-down and bottom-up approaches.
- **System testing:** Tests the **entire system** as a complete entity. Validates that the software works as intended in its **target environment**. Includes both **functional** and **non-functional** testing.
- **Acceptance testing:** Conducted to verify the system against user needs and business requirements. Usually the final phase before deployment. Often performed by the client or end-user.

Types:

- **Alpha Testing:** Conducted at the developer's site by internal staff.
- **Beta Testing:** Conducted at the customer's site under real-world conditions.

White Box Testing Strategy: Also called **structural testing**. Tests internal logic, code structure, loops, and conditions. Requires knowledge of the source code.

Examples:

- Statement coverage
- Branch coverage
- Path coverage
- Loop testing

8. Black Box Testing Strategy: Tests the **functionality of software** without knowing its internal structure. Focuses on **inputs and expected outputs**.

Techniques:

- Equivalence partitioning
- Boundary value analysis
- Cause-effect graphing
- Decision table testing

Validation Testing

Validation Testing is performed to ensure that the software meets the user's needs, expectations, and requirements.

It answers the question: "Are we building the right product?"

While verification ensures that the software is built correctly according to design, validation ensures it fulfills its intended purpose.

Objective

To confirm that the developed software:

- Satisfies all functional and non-functional requirements.
- Meets customer and business expectations.
- Is ready for delivery or deployment.

Debugging Technique:

Debugging is the process of identifying, analyzing, and removing errors (bugs) from a software program. It begins after testing detects a defect and continues until the issue is resolved and verified. In simple terms: Testing discovers that an error exists;

Debugging finds and fixes the cause of that error.

Objectives of Debugging

- To locate and correct the cause of defects or failures.
- To ensure the software functions correctly after correction.
- To prevent recurrence of similar errors in the future.
- To improve the reliability and stability of the software system.

Debugging Process:

Debugging is not a single-step activity but a systematic process consisting of several stages:

Step 1: Identify the Error

- Review test results or logs that indicate the failure.
- Reproduce the error under controlled conditions.

Step 2: Isolate the Source of the Error

- Analyze the program flow and data to pinpoint where things went wrong.
- Use tools like **debuggers**, **print statements**, or **log files**.

Step 3: Diagnose the Cause

- Determine the **root cause** — whether it's logic, syntax, data, or environment-related.

Step 4: Correct the Error

- Modify the code, configuration, or logic as needed.
- Ensure the fix does not introduce new defects.

Step 5: Re-test the Software

- Run the test cases again to verify that the issue is resolved.
- Perform **regression testing** if necessary.

Guidelines for Effective Debugging

1. **Understand the system thoroughly** before making changes.
2. **Reproduce the bug** consistently.
3. **Isolate the error** — minimize code area under investigation.
4. **Use debugging tools** instead of only print statements.
5. **Fix one error at a time** — don't rush multiple changes.
6. **Retest thoroughly** after corrections.
7. **Document the debugging process** for future reference.
8. **Perform regression testing** to ensure no new bugs are introduced.

Software Product Metrics

In software engineering, the success of a product depends not only on its functionality but also on its **quality attributes** like reliability, performance, maintainability, and usability. To measure and control these attributes, we use **Software Metrics**.

Classification of Software Metrics

Software metrics are generally classified into three categories:

Type	Definition	Examples
1. Product Metrics	Describe the characteristics of the software product itself.	Size, complexity, functionality, reliability
2. Process Metrics	Describe how the software development and maintenance process is carried out.	Defect removal efficiency, error detection rate
3. Project Metrics	Describe project management aspects.	Effort, cost, schedule, productivity

Metrics for analysis, design, source code, testing & Maintenance

Analysis Metrics: The analysis phase focuses on understanding user requirements and defining system functionality. Metrics in this phase aim to assess the completeness, correctness, and stability of requirements before moving forward to design.

One of the most widely used analysis metrics is the **Function Point (FP)** metric. It measures the functionality provided to the user based on the number of external inputs, outputs, inquiries, files, and interfaces. Function points help estimate the effort, cost, and duration required for development, regardless of the programming language used.

Another important measure is the **Requirements Stability Index (RSI)**, which indicates how much the requirements have changed over time. A higher RSI means that the requirements are stable, while a low RSI suggests frequent changes and uncertainty. Similarly, the **completeness of specification** metric measures how fully the requirements have been defined and documented, ensuring that all user needs are captured.

The **correctness of requirements** metric measures how accurately the documented requirements reflect the customer's needs. This can be determined by comparing the number of validated requirements to the total number of identified requirements. Finally, the **traceability metric** ensures that every requirement can be traced to its corresponding design and test cases. Good traceability reduces the risk of missing functionality during later phases.

Design Metrics: Design metrics evaluate the quality, efficiency, and modularity of the software design before implementation begins. These metrics focus on how well the design supports future maintenance, scalability, and reuse.

The **Design Size in Classes (DSC)** measures the number of classes in an object-oriented design and gives an estimate of the overall size and complexity of the system. Another useful metric is the **Number of Hierarchies (NOH)**, which measures the depth of inheritance structures. Deeper hierarchies can promote reuse but may also increase complexity if not properly managed.

Cohesion and **coupling** are two fundamental measures of design quality. **Cohesion** refers to how closely related the functions within a module or class are. High cohesion (measured using the Lack of Cohesion of Methods—LCOM) means the module has a well-defined purpose and is easier to maintain. **Coupling** (measured using the Coupling between Objects—CBO) measures the degree of interdependence between modules or classes. Low coupling is desirable because it reduces the ripple effect of changes and improves modularity.

Other design metrics include **design complexity**, which assesses the number of relationships and dependencies among components, and **encapsulation metrics**, which evaluate the degree to which data and behavior are hidden within modules. Effective encapsulation improves information hiding and reduces vulnerability to external changes.

Source Code Metrics: Once the design is implemented, source code metrics provide insights into **code quality, complexity, readability, and maintainability**. These metrics are essential for managing large codebases and predicting maintenance effort.

The most basic measure is **Lines of Code (LOC)**, which counts the total number of executable lines. Although simple, LOC is useful for estimating effort, productivity, and defect density. However, it should be used carefully since a higher LOC does not always mean better functionality.

A more meaningful metric is **Cyclomatic Complexity (CC)**, proposed by Thomas McCabe. It measures the number of independent paths through the program's control flow. Programs with high cyclomatic complexity are harder to test and maintain, as they contain more decision points.

Comment Ratio measures the proportion of comments in the code, reflecting documentation quality. Well-documented code is easier to understand and modify. **Halstead's Metrics**, another set of measures, evaluate program complexity based

on the number of operators and operands in the code. These metrics can be used to estimate program effort, difficulty, and even the potential number of bugs.

A composite measure, the **Maintainability Index (MI)**, combines several code metrics such as cyclomatic complexity, lines of code, and Halstead volume into a single value. A higher maintainability index indicates that the code is easier to modify and less prone to errors.

Testing Metrics: Testing metrics evaluate the effectiveness, efficiency, and coverage of the testing process. They help determine how thoroughly the system has been tested and how many defects remain undetected.

Test Coverage is one of the most important metrics, measuring the extent to which code, functions, or requirements have been tested. High coverage means a greater portion of the system has been verified, reducing the risk of hidden defects.

Defect Density measures the number of defects found per unit of code (usually per thousand lines of code or per function point). It gives an indication of code quality and helps compare different modules or versions. Closely related is **Defect Removal Efficiency (DRE)**, which measures the percentage of defects detected during testing compared to the total number of defects (including those found after release). A higher DRE indicates an effective testing process.

Test Execution Rate measures how many test cases are executed per unit of time, helping assess testing productivity. **Mean Time to Detect (MTTD)** and **Mean Time to Repair (MTTR)** measure the average time taken to find and fix defects, respectively. These metrics indicate the responsiveness and efficiency of the testing and debugging processes.

Maintenance Metrics: After the system is deployed, maintenance metrics measure how well the software performs over time and how efficiently it can be modified or repaired. These metrics are essential for assessing software reliability, stability, and maintainability in the long run.

Mean Time to Failure (MTTF) measures the average time the system operates before encountering a failure. A higher MTTF indicates greater reliability. **Mean Time to Repair (MTTR)**, on the other hand, measures how long it takes to correct and recover from a failure. Together, these two metrics provide insights into system availability and robustness.

The **Change Request Rate (CRR)** measures how often users request modifications or improvements. A high CRR may indicate evolving user needs or

poor initial requirement analysis. The **Maintenance Effort** metric quantifies the amount of time and cost spent on maintaining the software, including debugging, updating, and enhancing the system.

Finally, the **Impact Analysis Metric** evaluates how much of the system is affected when a change is made. A well-designed system will have limited impact from individual changes, showing good modularity. **Software Reliability** itself is often measured as the probability that a system will operate without failure for a specified period under given conditions. Reliable software enhances user trust and reduces maintenance costs.

Software metrics serve as **quantitative indicators** that help assess the quality and performance of software throughout its life cycle. Together, these metrics provide a structured approach for **monitoring, controlling, and improving software quality**, ensuring that the final product meets user expectations and performs reliably over time.

Important Question (Assignment III)

- Q.1: Explain software design process.
- Q.2: Explain software design model and software architecture.
- Q.3: Explain UML and structural modeling techniques.
- Q.4: Explain software testing strategies.
- Q.5: Explain software product metrics in detail.